

Principles of Programming Languages, 2025.07.03

Important notes

- Total available time: 2h (*multichance* students do not need to solve Exercise 1).
- You may use any written material you need, and write in English or in Italian.
- You cannot use electronic devices during the exam: every phone must be turned off and kept on your table.
- You cannot use library functions not covered in class in your code.
- Each exercise is valued 10 points, +1 bonus for the best solutions.

Exercise 1, Scheme

Define a function which takes a list (containing lists of any depth), and checks if every list and sub-list contains its own length. E.g. (*contains-length? '(3 (2 3) (2 (1 2)))*) should return #t, while (*contains-length? '(3 1 (2 3) (1 (2)))*) is #f.

Exercise 2, Haskell

Hamming numbers are the numbers whose only prime divisors are 2, 3, and 5. Formally, a Hamming number n is an integer of the form $n = 2^i 3^j 5^k$, for i, j, k natural numbers.

Define the infinite list of Hamming numbers, with type *hamming* :: [Integer].

Exercise 3, Erlang

Define a *parallel filter and map* operation, which has, together with the function for the map, two predicates (respectively called *pre-condition* and *post-condition*) for filtering the list values: the first one filters the values to be processed (i.e. it does not map the values that falsify the pre-condition), while the second one is used to filter the results.

E.g.

```
filtermap(  
  fun (X) ->      % pre-condition  
    X > 3  
  end,  
  fun (X) ->      % map operation  
    X * X  
  end,  
  fun (X) ->      % post-condition  
    X < 20  
  end,  
  [1, 2, 3, 4, 5]).
```


should return [16]

Solutions

Ex 1

```
(define (contains-length? l)
  (and (list? l)
        (member (length l) l)
        (all? contains-length? (filter list? l))))

(define (all? pred lst)
  (null? (filter (lambda (x)
                    (not (pred x))) lst)))
```

Ex 2

```
-- trivial version:
isHamming :: Integer -> Bool
isHamming n = red n == 1
  where
    red x
      | x `mod` 5 == 0 = red (x `div` 5)
      | x `mod` 3 == 0 = red (x `div` 3)
      | x `mod` 2 == 0 = red (x `div` 2)
      | otherwise      = x

hammingSimple = filter isHamming [1..]

-- more efficient version:
merge xs@(x:xt) ys@(y:yt)
  | x < y    = x : merge xt ys
  | x > y    = y : merge xs yt
  | otherwise = x : merge xt yt
merge [] ys = ys
merge xs [] = xs

hamming = 1 : merge (map (2*) hamming)
                  (merge (map (3*) hamming)
                          (map (5*) hamming)))
```

Ex 3

```
runit(Proc, F, Post, X) ->
  V = F(X),
  case Post(V) of
    true ->
      Proc ! {self(), V};
    false -> Proc ! {self(), none}
  end.

recv(none) ->
  none;
recv(Pid) ->
  receive
    {Pid, V} -> V
  end.

filtermap(Pre, F, Post, L) ->
  W = [case Pre(X) of
        true ->
          spawn(?MODULE, runit, [self(), F, Post, X]);
        false ->
          none
      end || X <- L],
  lists:filter(fun (X) -> not (X == none) end, lists:map(fun recv/1, W)).
```